rrent tier and feature access.

For example, if a customer is on Premium, we will base their health on Premium-level features to understand their level of adoption. If their health is red or yellow, it signifies risk. If green, it can signify expansion or flat renewal.

Leading and lagging indicators

Some metrics are more leading or lagging indicators. While we will lean toward a predictive solution, lagging metrics are incorporated to assess past performance.

The following graph (Early Warning Segmentation Framework) is used to provide a framework for which strategy to use and which resources to leverage. Customers are grouped by their Account Health and growth potential. Renewal Operations Analysts will support the Field in triaging accounts to identify where to spend their time.

Methodology

Health scoring

Starting point

The first approach was a calculation of multiple metrics to create a "black box" approach. This was neither helpful to the end user (CSMs, SAs, sales reps), it was not easy to understand the calculation, the Gainsight logic was inadequate, and was not action-oriented to know which aspects of the use case were great and which needed improvement.

PROVE Components

Category	Health Measure	Example	Why?	Metrics	Account Type	Maturity
Product	License Activation	The customer has assigned all licenses	Has the customer deployed their licenses? This is an indicator of seat reduction / expansion	License utilization	All	100%
Product	User Engagement	73% of users are Monthly Active Users	Are users logging in and using the product?	Unique Monthly Active Users / billableusercount	All	100%; in progress
Product	Adoption (Use Case)	Use Case adoption	Is the customer adopting use cases and progressing into "stickier" areas of GitLab?	SCM -> CI -> DevSecOps adoption	All	100%
Risk	CSM Sentiment	The sentiment as determined by the CSM, if applicable	What has the CSM determined from cadence calls?	CSM Sentiment	CSM owned	100%

Outcomes	ROI Success Plan	Ensure the ROI Success Plan is aligned to customer	A missing or poorly constructed Success Plan highlights a lack of alignment between GitLab and customer desired outcomes.	Green Success Plans Delivered EBRs	CSM owned	100%
Outcomes	Positive Business Outcomes (PBOs)	Completed Success Plan Objectives	Failed or missed PBOs can be a sign of distress; successful PBOs can highlight renewal expansion	Successfully completing at least one PBO each year	CSM owned	Not started
VoC	Support - Escalations	Emergency support tickets	Emergency support tickets can indicate unhappiness or frustration	Measure if there are Emergency support tickets in the last 90 days	All	100%
VoC	Support - Engagement	Customer sends in tickets	Determining if the customer is engaged with Support	Retain existing methodology, but tweak to allow more tickets as a good thing	All	70%
VoC	Support - CSAT	Customer completes CSAT surveys and provides feedback	Is the customer giving feedback and what are the scores (response + outcomes)	Benchmark a minimum XX% response rate for green health and provide CSAT results to CSM	All	Not started
VoC	NPS Surveys	The customer responds to and provides high scores	Because surveys are a good indicator of the customer's perception of the product and company; this can	Survey responses rates + survey scores	All	Not started
Engagement	Engagement	Recency of CSM cadence call	Lack of customer engagement	Date of last CSM cadence call	CSM owned	100%
Engagement	Executive Sponsorship	Are stakeholders aligned and communicating?	Lack of alignment and communication can indicate a disconnect between execs and ROI	Recency of aligned stakeholder communication	CSM owned	Not started
Engagement	Events	Is the customer attending GitLab events?	Event attendance indicates customer engagement, dialogues with team members, and face-to-face interactions	TBD	All	Not started
Engagement	Certifications	Are users within the account taking certifications? Are they maintaining their certifications?	Obtaining GitLab certifications is a positive for us and the customer; it also indicates their involvement in GitLab, knowledge of using GitLab, and provides an inference as an internal champion	TBC	All	Not started

Predictive Analytics

Predictive Analytics is not a silver bullet. It will not cure all that ails you. Instead, this methodology is probabilistic and incorporates health measures to correlate the typical journey of "healthy" customers (expand and renew) with "unhealthy" customers (downgrade and churn). For example, a healthy sales pipeline has few pushes (moving the close date) and progressively moves through stages (not stale). Conversely, an opportunity with multiple pushes and stuck in stages for long periods of time is an indicator of risk.

Prediction Type	Model Name	Status	Description
Expansion	Propensity To Expand (PTE)	Active	Predicts whether an account is likely to

expand (increase ARR) |
| Churn and Contraction | Propensity To Churn or Contract (PTC) | Active | Predicts whether an account is likely to churn or contract (decrease ARR) |

Appetite and ability to expand (Seats, uptier)

Triggers will be used for different events:

- A call to action to prevent downgrade because of lack of license utilization
- Heavy CI usage indicates they are meeting their business outcome and are ready to talk about the next stage, or
- The customer has very little growth but is successful and we should aim for a flat renewal.

Each of these metrics will be used to guide the account team in knowing when a customer is approaching the next or has met their milestone. The items listed below are examples of what an account team could look at to glean insights for a productive customer conversation.

Seat expansion

License utilization

1. Rapidly consuming licenses (measuring growth rates)
2. 90% license utilization

User activity

1. High UMAU (above 80%)

Uptier

- Desire for guest users
  - They purchased a high number of Premium licenses but could move many to Guest
- Consuming Free/Premium features that lead to Ultimate
  - DevSecOps
  - Agile Planning
- Success Plan objectives are aligned with Ultimate-level feature sets
  - DevSecOps
  - Agile Planning

Seat reduction

- Less than 75% license utilization (excludes onboarding customers)
- Consistent reduction in activated user count (number of deactivated users M-o-M)
- CSM renewal risk == Seat Loss

Downtier

- Not using Ultimate-level features
  - DevSecOps
  - Agile Planning

- Success Plan objectives not aligned with Ultimate-level feature sets
- CSM renewal risk == downtier

 Churn

Indicators from Seat Reduction or Downtier above plus:

 Customer experience

- Customer has gone dark
- Loss of internal champion/stakeholder
- CSM renewal risk == churn

 Renewal opportunities

- Stages not progressing
- Pushes
- Lack of Oppty customer activity

 References

- MVC Early Warning System epic
- Customer Analytics Roadmap (internal-only document) (slide deck)
- Customer Success Services (client facing)
- Operational Data Vision
- Cloud Licensing Documentation (internal handbook)
- Strict Cloud Licensing (internal handbook)
- Service Ping Metrics list (subscription, operational, and optional)
- Operational Service Data (internal handbook)
- Operationalized Usage Metrics in Gainsight/Tableau (Metric Dictionary)

SECTION: customer-success/customer-onboarding/index.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

 GitLab Quickstart for New GitLab.com Customers

Congratulations on your new GitLab subscription! Here are some steps to complete to your onboarding as a new customer is successful. Consider this your hub for getting started.

 Key Terminology

Term	Definition
GitLab.com	Multi-tenant SaaS, subscription-based GitLab platform
Namespace	One place to organize different projects, can be a group or personal namespace
Group	Allows you to manage multiple projects and sub-groups
Project	A source code management (SCM) repository

| Member | User who has access to your groups and projects |
| Customer Portal | Place to complete tasks around account management, such as purchasing more seats or CI/CD minutes |

Sign Into the Customer Portal

You can sign into the Customer Portal with your GitLab.com account. This is what you use to sign into GitLab.com.

(Optional) Update Subscription and Billing Contacts

- Subscription Contact: The subscription contact is the primary contact for your billing account.
- Billing Contact: The billing contact receives all invoices and subscription event notifications.

Sometimes you may want another user to be the primary adminstrative owner of the GitLab subscription (Subscription Contact) or receive invoices (Billing Contact). You can edit the subscription and billing contacts by following these instructions.

Link Your Namespace to Your Subscription

In order to use your new GitLab subscription, be sure to link the subscription to your group-type namespace.

Follow the instructions here to link your subscription to a group.

Confirm the linkage by navigating to your group and selecting Settings > Billing.

(Paid Customers Only) Confirm Support Entitlement

There are a few ways an account may be created for you in the Support Portal. To access the Support Portal, first attempt to reset your password to see if an account was created for you. This may have happened on the backend if you are the named contact of your subscription, if you submitted a ticket via email, or if someone else from your organization added you as a support contact. Follow the reset instructions and then sign in.

If there is no existing account, create one manually and then sign in.

You can access the support portal here.

Manage Support Contacts

Via ticket

Add Support Contacts via ticket by clicking the Support portal related matters form and selecting the problem type Manage my organization's contacts. This has a limit of 30 contacts per Organization. You can add contacts via a new ticket with the same Support portal related matters form.

Via contact management project (CMP)

Alternatively, request a CMP to be created. This project will manage your support contacts via a file called contacts.yaml that you can edit to add or remove support contacts. To get started, use the Support portal related matters form and select the problem type Contacts management project setup/questions. Please note if you choose to disable the CMP in the future it cannot be reenabled.

You can read more about managing support contacts here.

Create GitLab Groups

It is recommended at this point to define your group structure. Groups are collections of projects, and projects are individual repositories that can act as standalone services or microservices.

Membership should also be considered at this point, as memberships are inherited by default from the top level group in nested sub-groups and projects therein.

Recommended Group Structure Configuration

We have a few suggested models detailed here on how to organize your groups based on your organization's needs. Some recommendations may include structuring groups by business unit, client, or functionaltity/application.

Keep in mind as you define your group structure that membership can be updated to be more permissive at each sub-group's level via direct memebership.

(Optional) Import Projects from External SCM Sources

If you are migrating to GitLab from a different SCM tool such as BitBucket or GitHub, you can do so in a few ways, each with different considerations.

We list our supported imported sources here.

You can also leverage GitLab Professional Services to aid in migration via a tool called Congregate. Contact your Account Executive for more information on migration services.

You can also migrate groups and projects between GitLab instances (for example, from GitLab self-hosted to GitLab.com) via Direct Transfer subject to rate limits detailed here.

Provision Users

Instead of manually adding users by invitation, you can configure SAML SSO for your GitLab top-level group. SCIM allows for the automatic addition and removal of users to the GitLab group as well as subgroups and projects.

Please note that deactivating the user on the identity provider does not deactivate users on GitLab.com, it removes their access to the group.

Resources for configuration on GitLab.com:

- SAML SSO for GitLab.com groups
- Configure SCIM for GitLab.com groups

Permissioning

Familiarize yourself with the Permissions of GitLab listed here. It is recommended to follow the principle of least privilege when providing access to users.

If you would like users to automatically have read-only access to not only the top level group, but sub-groups and projects, it is recommended you set the default role to Guest. Note that Ultimate customers have an unlimited number of Guest users. You can set the default role in Settings > SAML Single Sign On Settings > Configuration.

If you would like to restrict users from having access to any sub-groups or projects without explicit invitation, it is recommended you set the default role to Minimal Access. Note that Minimal Access users are billable.

Define Custom Roles

There are cases where you may want to customize the role a user can have. You can create a Custom role with any permissions listed here expanding on a base role.

Please note you must be the Owner of the group to define a custom role. You can assign the custom role via UI or API to an existing user.

Subscribe to Status Updates

You can monitor the status of GitLab.com at status.gitlab.com. You can subscribe for updates by clicking the Subscribe button in the top right of the page, and receive updates via email, webhook, RSS, and more.

Additonal Resources

- Professional Services Catalog
- Get Started for Enterprise
- Customer Portal Documentation
- GitLab Adminstration on SaaS Webinar

SECTION: customer-success/customer-success-vision/index.md

"Customer Success is when customers achieve their desired outcome through interactions with your company with the appropriate experience." - Lincoln Murphy

Objective

Create a company-wide customer success approach, providing an engagement framework for the

Customer Success organization and integrating related programs and operations from GitLab operations (i.e., marketing, sales, customer success, product / engineering and support).

Goals

Deliver faster time-to-value and customer-specific business outcomes with a world class customer experience, leveraging the full capabilities of the GitLab application. Increase net dollar retention as well as lifetime value (LTV).

High-Level Visual of Customer Journey

Video Introduction

Video Introduction to Customer Journey and Process Framework

A pdf of Customer Journey

High-Level Visual of GitLab Adoption Journey

Capabilities Roadmap

The following shows the high-level view of the capabilities that we will be developing as mature our customer success team, processes and systems. A pdf version for viewing.

Strategy and Priorities Page

The Strategy and Priorities Page (in Gdrive, search for "FY24 Customer Success Priorities and Capabilities" - this is internal only) will show updates on our strategy and priorities for the given period.

Product Usage Data

Product usage data is key to driving customer and GitLab outcomes by providing visibility to customers' adoption of licenses, use cases, and capabilities. For more about how we use and plan to use this data, please see our "Product Usage Data Vision" page.

Lifecycle Stages

Each customer deployment will go through the following lifecycle stages.

Onboarding: The objective is to prepare the customer for a successful customer journey with GitLab, meaning they can achieve their business outcomes with a great experience. This includes success planning, expectation setting on engagement approach and tools, and education on GitLab resources, programs and support services. Onboarding is completed when all the tasks are completed or closed (i.e., not applicable).

Implementation: The objective is to ensure the customer has the right infrastructure to support GitLab solution operations. For self-managed customers, this could include setting up on-premises equipment and/or cloud infrastructure. For customers leveraging GitLab.com, this includes integration of the GitLab cloud service with the customers environment (e.g., SAML SSO integration). This is considered complete when production infrastructure is ready for use.

Adoption: The objective is to support our customer's utilization of the GitLab solution to address the customer's original purchase intent (i.e., use case(s) and capabilities, licenses). Adoption is complete when:

1. 80% of the licenses from the original purchases are activated
1. the customer is successfully adopting the capabilities or use cases from their original purchase intent.

These will be measured according to product analytics (if available) or through agreement with the customer.
We define the adoption of a use case using the criteria established in our Customer Use Case Adoption page.

Optimize and Grow: The objective is to enable the customer to get additional value from the GitLab platform. This is achieved through the adoption of additional features, use cases and/or stages, deeper process and operational integration in a customer's environment, optimization of application performance and availability, expansion into additional teams, and additional application of GitLab and DevOps best practices. The customer will remain in this stage as long as the customer continues to renew. The maturity of the customer will be tracked by product analytics (if available) or by collecting feedback from the customer on use cases adopted.

Measurement and KPIs

Time-to-Value KPIs

As part of our customer journey, we highly value the customer's initial experience and measure time-to-value. Specifically, we will measure the time in calendar days from the initial transaction to:

- Engagement: Represents our time to engage the customer. Completion defined when the CSM has their first meeting with the customer.
- Onboarding: Completion is defined when all the onboarding tasks are done.
- Infrastructure Ready: Completion is defined as either 1) production implementation is complete for on-premises installations 2) GitLab.com is integrated into a customer's environment or 3) Cloud and on-premises environments ready for a hybrid deployment
- First Value: Represents a small subset of users are using the product in a Production environment. This is achieved when a customer activates 10% of their licenses.
- Outcome Achieved: Represents delivery to original purchase intent. We want to capture the delivery of outcomes against their original purchase intent. New and changed goals will continue to be tracked with the engagement, but not included in this milestone.

Customer Health

To improve the customer experience, deliver on customer outcomes, and increase net ARR, GitLab

is focused on measuring and improving Customer Health. Customer Health includes multiple health measures that, when aggregated, gives a holistic view into the customer and allows drill-down into specific areas. We have created and adopted the PROVE Value approach:

- Product: License + User Engagement + Use Case
- Risk: CSM Sentiment + Opportunity Renewal risks
- Outcomes: Success Plan + Verified Outcomes
- Voice of the Customer: Support + Surveys
- Engagement: Customer Engagement + Executive Sponsorship + Events + Certifications

The intention, both of the P.R.O.V.E. components and overall philosophy, is that we need to "prove value" to the customer.

For a detailed description of Customer Health and Early Warning System methodology, see Customer Health Scoring. This will include the methodology around how we score the health of accounts, along with how we enable the team toward a proactive renewal approach.

Retention and Reasons for Churn

We measure customer success through Gross and Net Retention.

Reasons for Churn / Expansion, Dollar Weighted

A measure of the causes for retention (compared to the same time period for the previous year) MRR decreases (churn) or increases (expansion). Churn is specified as Cancellation or Downgrades. Expansion is specified as Seat Expansion, Product Change, Product Change/Seat Change Mix, or Discount/Price Change. These are reported as a percentage using the change in MRR for the given reason over the total MRR change for all types in either the Churn or Expansion category. Trueups are excluded from these metrics.

Professional Services Standard Cost

We use a standard cost estimate to project margin on PS Statements of Work. The standard rate is calculated by dividing the average annual OTE plus benefits by the estimated annual billable hours. For this calculation, we assume 1,880 billable hours annually. We update the standard cost estimate on a quarterly basis.

To standardize the cost estimate for projects, make a copy of the SOW Cost Estimate Calculator and attach it to the issue for the SOW in question. This calculator will be updated to reflect any changes to the standard cost estimate.

If you need help with determining the standard cost rate or if it is applicable to your project, please contact your Finance Business Partner.

Scope

- Processes, procedures, metrics and tools / systems for the Customer Success team
- Integration of the related processes and operations for other GitLab business groups

Deliverables

1. Provide a process blueprint for the customer success engagement processes, including:
   - High-level customer journey (i.e., single stage and stage expansion)
   - High-level multi-year engagement summary
   - Processes, procedures and detailed flowcharts
   - Customer and GitLab-centric metrics
1. Framework structure that will address specific differences and needs based on:
   - Customer segmentation (Large, SMB, Commercial)
   - Product differences (e.g., stage-specific, use case and/or feature playbooks)
   - Customer personas (e.g., executive sponsor, tools/infrastructure executive, development executive, etc.)
1. Integrate processes and operations for direct customer engagement processes such as:
   - Marketing: messaging alignment and consistency, journey integration with marketing stages (e.g., awareness), customer advocacy, advisory boards, collateral development, digital journey (i.e., tech touch, hybrid digital/CSM engagement)
   - Sales: account planning and strategy, POC, success planning, IACV, renewal forecasting and execution, training and enablement
   - Product / Engineering: Product analytics and data insights, UX journey map alignment, in-app adoption (e.g., product-led onboarding), application/stage/use case/feature adoption, "voice of customer" reporting, escalations and defect and enhancement requests
   - Support: Customer health, escalation processes
   - Finance: Financial metrics and targets (e.g., margins), budget and forecasting
   - People Operations: Job types, grades and families
   - Information Technology: Customer and operational dashboards, workflow management capabilities (i.e., SFDC, customer success platform), journey automation

Process Framework

SECTION: customer-success/demo-systems/_index.md

Overview of Demo Systems

The GitLab Demo Systems provide infrastructure for the GitLab Customer Success, Marketing, Sales, and Training teams to demonstrate GitLab features, value propositions, and workflows in a variety of asynchronous and live capacities.

The Demo Systems were originally architected by Jeff Martin starting in October 2019 when he was a Senior Demo Systems Engineer. Since Jeff moved to the IT team in June 2021, Logan Stucker (Demo Architecture) and Scott Cosentino (GitLab University) have taken over as the primary maintainers of the Demo Systems, including supporting training instructors and students with GitLab Learn Labs.

For questions about what demo sample projects are available or peer assistance with troubleshooting your failed pipeline job, please ask in the demo-architect-partners Slack channel.

Please tag @Jeff Martin in one of the following Slack channels with any questions or requests

related to infrastructure or access requests.

- demo-systems is for SA, CSM, and PSE team members with questions or needing technical assistance. No longer for training/workshop related posts.
- demo-architect-partners is for workshop-related discussions.
- demo-systems-ps-education is for ILT/SPT/etc related discussions for Professional Services.
- sandbox-cloud-questions is for help and support with the Sandbox Cloud (AWS Accounts and GCP Projects).

Please consider this handbook documentation to be the single source of truth ("SSOT") for all resources that use the gitlabdemo.com, gitlabdemo.cloud, and gitlabtraining.cloud domain names.

Why do we have demo systems?

- Why shouldn't we just use GitLab.com? Although you can use GitLab.com for showing most of the value of GitLab use cases, there are some administrative features that require the deployment of GitLab Omnibus infrastructure in AWS, GCP, or local VM/container. Many of our enterprise customers opt for self-managed over GitLab.com so we are mindful of "showing the customer what they'll see in production".

- What's special about our infrastructure? The demo systems infrastructure doesn't do anything special that a customer or partner company couldn't do themselves with the appropriate staffing and engineering investment.

- Are there special engineering or scalability considerations with training classes and workshops? Yes. The GitLab product was designed for users to be doing different activities throughout the day and the smaller reference architectures are not designed for dozens or hundreds of users to click the same button and running the same background jobs or pipeline jobs at the same time. Our users are also ephemeral and have automated garbage collection requirements that are not customary for conventional GitLab product use cases. This requires special scalability considerations, notably with the Container Registry, Sidekiq, and Kubernetes that we have to accommodate for. Here's some of the things we're looking for with scalability challenges.

  - Autoscaling runners for 500 simultaneous pipelines started in 10 seconds
  - Autoscaling Kubernetes nodes for 500 simultaneous review apps/deployments in 60 seconds
  - Auto DevOps pipelines that consume lots of wasted resources
  - Kubernetes services that are not needed (ex. Postgres database)
  - Intensive test jobs that are not needed during workshop (ex. Code Quality, Dependency Scanning, etc.)
  - Project export/import queued job fails with 500 simultaneous project imports
  - Features in your project that have known issues with import/export process (ex. wikis)
  - Administrative access for students (alternative use cases)
  - Package registry caching and garbage collection
  - Container registry caching and garbage collection
  - CI images pulling from Docker Hub with rate limits
  - CI image versions that are incompatible or have been upgraded with bug fixes
  - Using templates in .gitlab-ci.yml without realizing the underlying job load.
  - Using custom .gitlab-ci.yml files without comments of the actions being performed
  - Dependency proxy configuration (particularly for npm and maven dependencies)

- Lack of step-by-step instructions that leads to student misconfigurations and errors

Shared Environments

These shared environments are referred to as the Demo Cloud or Training Cloud. Historically, training users used the Demo Cloud so the names are used interchangably in some conversations.

{{% panel header="Demo Systems v1 Deprecation" header-bg="warning" %}}
The <code>gitlab-core.us.gitlabdemo.cloud</code> instance was deprecated on 2021-04-20 and destroyed on 2021-06-03. No data back up is available. See Access Shared Omnibus Instances instructions for accessing the <code>cs.gitlabdemo.cloud</code> instance (direct replacement).
{{% /panel %}}

- cs.gitlabdemo.cloud - This is the primary GitLab Omnibus instance that all team members have access to for creating groups, projects, and sandbox purposes on a self-managed Omnibus instance. Please keep in mind that this is a shared environment across all team members and you should treat the Admin areas as read-only.
- ilt.gitlabtraining.cloud - This is used for instructor-led training classes. You should generate credentials for this instance if you are an instructor and need admin access to be able to import sample projects and/or see the groups for all of the students in a class.
- spt.gitlabtraining.cloud - This is used for self-paced training classes that are published in EdCast. You should only generate credentials for this instance if you are involved with instructional design or certification grading of self-paced student courses. If you are enrolled in a self-paced training class, you should follow the instructions for redeeming an invitation code to generate temporary credentials that can be used for accessing the instance that has been pre-configured for the training lab guide steps.
- workshop.gitlabtraining.cloud - This is used for enablement and field marketing workshops that are delivered on a routine basis. You should generate credentials for this instance if you are involved creating lab sample projects, lab guides, presenting, or supporting a workshop.

Isolated Environments

- AWS Account: See the instructions for provisioning your own isolated AWS account with the GitLab Sandbox Cloud.
- GCP Project: See the instructions for provisioning your own isolated GCP project with the GitLab Sandbox Cloud.
- AWS Elastic Kubernetes Service (EKS) Cluster: You can use your AWS account to provision an EKS cluster using the Adding EKS clusters GitLab documentation.
- GCP Google Kubernetes Enginge (GKE) Cluster: Send a message to Jeff Martin with questions about clusters that are in the group-cs GCP project. See the tutorial for configuring GitLab with group-level Kubernetes cluster to add your cluster to your GitLab group.

How to Get Started

These instructions are for Demo Systems v2 that uses the gitlabdemo.cloud Portal. The Demo Systems v1 infrastructure that used gitlabdemo.com has been deprecated except for training class invitation codes.

Access Shared Omnibus Instances

These instructions provide you access to one or more of our shared environments (Omnibus self-managed instances).

> The Demo Cloud Portal and provisioning system is powered by gitlabdemo-cloud-app, an open source project created by Jeff Martin.

1. Visit the GitLab Demo Cloud Portal (https://gitlabdemo.cloud) and sign in with your Okta credentials.
1. Navigate to the Environments top navigation link or click on the View Environments button on the dashboard.
1. Locate the Environment Instance that you wish to access and click the Generate Credentials button.
1. After the credentials have been generated, click the View Credentials button.
1. Create a new record in your 1Password vault with your generated credentials.
1. Click the Access Instance button to open a new tab with the URL of the GitLab Omnibus instance.
1. Bookmark the Omnibus instance that you can use to easily access this in the future. You do not need to access the https://gitlabdemo.cloud portal each time you want to access the instance since this is only for access requests (credential generation).
1. After signing in, a group has been pre-created for you to store your projects under. To help with namespace consistency and security best practices, please do not create other top-level groups with custom names. You can create any subgroups or projects you'd like under your pre-created group or in your personal namespace.
1. Each instance is pre-configured with shared GitLab Runners and a Kubernetes cluster. These are for consumption purposes when running CI/CD pipelines and you do not have administrative access to these in the shared environments.
1. You can ask for help from other peers in the cs-questions Slack channel.

AWS Account or GCP Project (Sandbox Cloud)

See the Sandbox Realm handbook page for instructions on creating your own AWS account and/or GCP project that you can use for deploying your own infrastructure with the benefit of centralized billing.

Invitation Code Creation

Invitations codes are created by the demo systems team and can be requested by following the Workshop Preparation guide.

Invitation Code Redemption

> Warning for GitLab team members: This process uses an existing GitLab.com account so please ensure this is set up before hand.

1. Visit https://gitlabdemo.com and click the Redeem Invitation Code button.
1. Type in the 8-character invitation code that was provided by your instructor or in your course materials.
1. Press Redeem and Create Account.
1. Type in your GitLab username (without the @), then click Redeem and Create Account.
1. Click the blue My Group button to open a new tab with the URL of the your GitLab group

living on Learn Labs.

Workshop Preparation

> The workshop process has iterated multiple times. The latest version of the workshop preparation process has been simplified into a self-service model.

As the workshop presenter and supporting team member, you are responsible for importing and testing your sample projects and lab guide content. There is no support provided for misconfigured projects, failing pipelines and jobs, or GitLab Runner error messages specific to projects and jobs.

There is no more peer review or approval process other than scheduling coordination with the field marketing team. These instructions provide best practice guidance. If you need assistance, please ask in the demo-architect-partners Slack channel to get help from other team members that have delivered similar workshops.

Workshop preparation steps will be linked on the resulting issue after completing the request form here

Workshop lab guide catalog

All of the workshop content that are created officially can be found in the Learn Labs Sample Projects group. Often workshops don't fit a customers needs out of the box so we support creation of custom customer workshops/labs/demos through the DA Portal

Version Upgrades and Maintenance

We perform version upgrades on the weekend following the monthly release. The weekend upgrades are performed at a random time on Saturday or Sunday based on engineer availablility and lasts for approximately 30 minutes.

We delay the upgrade window for updates that we consider risky or occur during holidays. This occurs during May each year that aligns with the US Memorial Day holiday, in December around the Christmas Holiday, and in January at the end of the fiscal year when we have a configuration freeze until sales demos are completed.

For patch and security updates, we will usually only perform upgrades for critical updates and will announce maintenance windows in the demo-systems channel on Slack.

Legacy Version Support

We keep our shared environment up-to-date with the latest versions to help showcase the value that the newest features and solutions offer.

For demo and sandbox use cases requiring an older version, you can deploy a GitLab instance in a container in the Container Sandbox or using Omnibus in the Compute Sandbox. We do not offer any data migration or parity configuration support.

GitLab Duo features

GitLab Duo is enabled for the demo cloud environments. You may assign a seat to yourself & other users in the Admin settings.

Tutorials

- Configuring GitLab with group-level Kubernetes cluster
- Create a Jenkins Pipeline (Deprecated, Educational only)

Sample Data

Historically, there has not been a consistent set of demo data. Each of our Solutions Architects are responsible for creating their own demo data or forking projects from other team members.

See the handbook page for Demo Readiness and Existing Demonstrations to get started.

Please see the Solutions Architecture Initiatives issue tracker for more information on the crowd sourced OKRs that are in progress and the development of our Communities of Practice.

Projects and Code Repositories

These are the projects that make the Demo Systems possible behind the scenes. You are welcome to study and learn from any of our source code. Each project is classified as Public or Private depenending on the security risk of the source code or information contained within.

Demo Systems v2 (Deprecated)

- Public Underlying Terraform Modules and Ansible Role
  - terraform-modules
  - terraform-modules/gcp/gce/gcp-compute-instance-tf-module
  - terraform-modules/gcp/gke/gke-cluster-tf-module
  - ansible-roles/omnibus
- Public Assembled Terraform Modules and Environments
  - terraform-modules/gcp/gitlab/gitlab-omnibus-sandbox-tf-module
  - environment-templates/gitlabtraining-shared-environment-template
  - INSTALL.md example
- Private Environments IaC - see terraform/terraform.tfvars.json and CI pipeline
  - environments
  - environments/cs-gitlabdemo-cloud
  - environments/ilt-gitlabtraining-cloud-iac
  - environments/spt-gitlabtraining-cloud-iac
  - environments/workshop-gitlabtraining-cloud-iac
  - environments/app-gitlabdemo-cloud
- Public Management Applications
  - management-apps/gitlabdemo-cloud-app
  - gitlab.com/hackystack/hackystack-portal (Open source namespace)
  - sandbox-cloud/apps-tools/hackystack-portal (Mirror for Ultimate features)
- Private - Ops Sandbox Cloud Infrastructure

  - ops.gitlab.net/cloud-realms/master-account/gcp/gcp-hackystack-portal-prd-tf
  - ops.gitlab.net/cloud-realms/master-account/gcp/gcp-hackystack-portal-prd-ansible
  - ops.gitlab.net/cloud-realms/master-account/gcp/gcp-sandbox-cloud-dns-tf
- Private Runbooks
  - runbooks
  - ops.gitlab.net/cloud-realms/apps-tools/runbook-docs

Demo Systems v1 (Deprecated)

The Demo Systems v1 repositories can be found in gitlab.com/gitlab-com/customer-success/demo-systems.

- Private Terraform Monolith Environments and Modules
  - infrastructure/demosys-terraform
- Private Ansible Monolith Configuration and Roles
  - infrastructure/demosys-ansible
- Private Management Applications (gitlabdemo.com)
  - infrastructure/demosys-portal
- Issue Trackers
  - Demo Systems

Handbook Links for Related Infrastructure

- GitLab Sandbox Cloud
- GitLab Infrastructure Standards
- GitLab Infrastructure Standards - Labels and Tags
- Demo Systems Kubernetes Architecture Docs
- Demo Systems Network Architecture and Subnet Docs

Help and Support

We use Slack for real-time support and quick fixes. If in doubt of how to get help, ask in demo-systems. The issue trackers are used for tasks and projects that take longer than 30 minutes. We do not use email for internal team communications.

- Demo Systems Issue Tracker
- demo-systems Slack channel (for Demo Cloud announcements, questions, and technical support with Demo Cloud)
- demo-systems-ps-education Slack channel (for ILT/SPT training lab discussions)
- demo-systems-workshops Slack channel (for workshop discussions)
- sandbox-cloud Slack channel (for Sandbox Cloud announcements)
- sandbox-cloud-questions Slack channel (for Sandbox Cloud questions and technical support)
- demo-systems-admin@gitlab.com (for non-Slack users)

SECTION: customer-success/demo-systems/onboarding.md

Demo Systems Initial Set Up

Preface

If you experience any roadblocks while setting up your environment, open an issue in the original project or attend the office hours linked below.

The goal is that you will be able to use and contribute to this shared demo group: <https://gitlab.com/gitlab-learn-labs/webinars> by the time you complete this.

You can collaborate with your onboarding buddy and your colleagues to remove this blocker, and open a merge request to improve the instructions and make onboarding easier for everyone.

Demo Systems & Shared Demos Office Hours

Feel free to join our weekly office hours to ask any questions if you get stuck, linked at the top of this doc: Demo Systems Onboarding Links

Envionments

You have access to three types of environments for performing demos.

GitLab.com SaaS Groups

You will have your two of your own groups on GitLab.com that allow you to showcase featues with each SaaS license tier.

- https://gitlab.com/gl-demo-ultimate-{handle}
- https://gitlab.com/gl-demo-premium-{handle}

This group should then be where you store all of your demo projects as it will not be constrained by the limitations of trying to just keep your demos in your personal namespace (ex. Epics, Security Dashboard and other group features).

-] Action: Do not try to create these groups yourself. Open an access request using the [GitlabComLicensedDemoGroupRequest template.

Self Managed Omnibus Shared Instances

You have access to an always-on GitLab Demo Cloud self-managed GitLab Omnibus instance that you have admin rights to for showing all of the features of the Admin UI that is not shown in GitLab SaaS. This also provides a backup for demos if something goes wrong with GitLab.com.

When you provision your credentials, a new top-level group will be created with your name that you can store your projects in. This is a shared environment so please do not change any admin-level settings so you don't break another team member's demo.

-] Action: Follow the [self-service instructions to provision credentials for the cs.gitlabdemo.cloud instance.
- [ ] Action: Join the demo-systems Slack channel for system maintenance announcements and a safe space to ask questions if you have any problems.

Personal AWS Account and GCP Project

Each team member can use the GitLab Sandbox Cloud to provision their own AWS account or GCP project for deploying infrastructure yourself.

> For the purposes of onboarding, we will focus on GCP. You will not need your AWS account right away, so you can always come back later.

-] Action: Follow the [self-service instructions to provision a GCP project.
-] Action: Follow the [self-service instructions to provision an AWS account.
-] Action: Read about how [Terraform Environments have been automated with the Sandbox Cloud and explore the available templates. You can follow the self-service instructions to create an environment using one of the templates or create any resources manually yourself in your AWS account or GCP project.

Please note that services may take a few minutes before being ready. If you log in and immediately see an error, wait a few minutes then try to access the account again.

You can ask for help from other peers in the sandbox-cloud-questions or cs-questions Slack channel and tag @Logan Stucker.

Set Up the GitLab Agent for Kubernetes for Your Group

At this point you should have your own group on GitLab.com SaaS as well as a GCP project created using gitlabsandbox.cloud.

Task 1. GKE Cluster

> Our first task will be to set up a cluster on GKE.

1. Login to your Google Cloud console.
1. Check if the API for Kubernetes Engine is enabled.
1. Check if the API for Compute Engine is enabled.
1. Navigate to Kubernetes Engine > Clusters.
1. From here we want to click CREATE,
1. In the popup modal window click Configure next to the Autopilot: Google manages your cluster (Recommended) option.
1. Rename the cluster to a not objectionable identifiable name.
1. Choose the region closest to you.
1. Leave the rest of the settings as-is.
1. Click create. This will kick off the building of the cluster which will take around 10 minutes.

> While we wait for the cluster to spin up we are going to set up cluster management and a kubernetes agent connection in the ultimate group we set up in the first step. Let's start by navigating there.

Task 2. GitLab Agent for Kubernetes & Cluster Management

> This agent will enable you to pull any project into your group and deploy it out using the

agent we set up. If you want to set up for both GCP and AWS you will need to define two sub groups and have an agent + cluster management in each.

1. Navigate to your Ultimate group.
1. Click on New Project.
1. On the follow up screen click Create from template.
1. In the template options you are going to want to scroll down to find the GitLab Cluster Management project then click Use Template.
1. Name the project Cluster Management.
1. Keep it at Private visibility level.
1. Click Create project.
1. After waiting a few seconds for the project to be created we will then want to click Web IDE so that we can start setting up the GitLab agent for Kubernetes.
1. First click new file where you can then copy in the snippet below and click Create file: .gitlab/agents/primary-agent/config.yaml
1. Within the config.yaml we want to add the code below, where you edit the & values to be the correct paths to your project & group. Both of these values can be grabbed from the URL you have the Web IDE open in. If you get stuck feel free to reference this agent config.

```
yaml
gitops:
  Manifest projects are watched by the agent. Whenever a project changes,
  GitLab deploys the changes using the agent.
 manifestprojects:
  Adjust the line below for your Project
  For Example: - id: gitlab-learn-labs/webinars/cicd-demo-project
 - id: {path-to-your-group-and-project}
   paths:
   - glob: 'manifests/.yaml'

  The CI/CD tunnel is always enabled in the project where you register and configure the Agent.
  This connection can be shared with other groups and projects.
 ciaccess:
   groups:
    Adjust the line below for your Project
    For Example: - id: gitlab-learn-labs/webinars
   - id: {path-to-your-group}
```

> Notice how the path example above does not include https://gitlab.com but only the path after it
>
> This config is telling the agent to look at our Cluster Management project for any manfiest changes as well as to allow any other projects within our group to use it for deployments.

1. Once your agent config is set up, go ahead commit the changes to the main branch.
1. Once committed, we can click our project name in the top left to get back to our project overview page.

1. From the landing page use the left hand navigation menu to click through Infrastructure > Kubernetes clusters.
1. We then want to click Connect a cluster.
1. In the popup modal window select our new agent, primary-agent, from the dropdown and click Register.

> Throughout your demos you may get some questions about the differences between the GitLab agent for Kubernetes and our old certificate approach. In short, the agent is far more secure and offers alot of flexibility, but it is a good idea to research this further for when the question comes up.

1. In the resulting popup we want to copy the helm command somewhere locally to reference later. Now that we can connect our agent we also want to navigate back to our Google Cloud console to see if our cluster has been created yet.

Task 3. Connecting the Agent

1. If the cluster is ready to go you will see a green checkmark in the status column. If green lets go ahead and click into the cluster.
1. On the resulting page click CONNECT.
1. Click RUN IN CLOUD SHELL. Wait for the shell to spin up.
1. Then run the from GCP auto generated command to connect to the right cluster. If a popup comes to authorize, press authorize.
1. Copy and paste the helm command that we got from setting up our agent in GitLab into the gcloud terminal.
1. We then want to navigate back to our GitLab.com tab where we can close the agent connection popup if it is still open. Go ahead and refresh the page to make sure that the agent has made the connection.
1. In the GCP Cloud Shell still connected to your GCP C:uster, you can view the logs with this command: kubectl logs -f -l=app=gitlab-agent -n gitlab-agent-primary-agent
1. Next using the bread crumbs at the top of the page navigate out to the top level of your group. Ensure that you are at the group level and not project level or the rest of the pipelines will fail. We then want to use the left hand navigation menu to click through Settings > CI/CD.

> Not adding the variables to the group level is the most common mistake people make when trying to set this up. Take your time and read carefully.

1. In the CI/CD settings, locate the Variables section and expand it.

1. Click Add variable and use the key value pair example below to add the variable.

text
KUBECONTEXT:{path-to-your-group-and-project}:primary-agent

text
Example
KUBECONTEXT:gitlab-learn-labs/sample-project-testing/cm-test:primary-agent

> This variable is used for any projects that have access and want to know where to look for the agent config. Without it, the Cluster Management project will fail because it dosent know what agent changes it is meant to compare to.

1. Unselect Protected and Masked.
1. When done click Add variable
1. Next we want to click the name of our group in the top left then click back into our cluster management project.
1. Once again we will click Web IDE to make some code changes.
1. Once in the Web IDE click into the helmfile.yaml.
1. Uncomment line 19. - path: applications/ingress/helmfile.yaml

> You can come back later to set up cert manager as well. Just keep in mind that you will need to set up your own domain name or use sslip.io due to cert limitations with nip.io.

1. Once done go ahead and commit the change to master. This time you should see a pipeline kick off in the bottom left of the webpage. We want to click the hyperlink that starts with in the bottom left to get a view of the cluster management pipeline.

> Line 19 holds the command to install and ingress controller in the cluster. By doing this GCP allows our application to be accessible through a public IP

1. Wait a few minutes for the first job to complete. If everything was set up correctly the job will pass and then we can manually start the sync job. If the job fails, reference the troubleshooting section before moving on.
1. Once the pipeline has fully run (~5 minutes), go back to the tab we have the Google Cloud console open on and in the gcloud terminal run the command below.

```
bash
kubectl get service --all-namespaces
```

1. Within the gitlab-managed-apps namespace, look for the service named ingress-ingress-nginx-controller. We then want to copy its EXTERNAL-IP value and use it locally.
1. Now that we have the ingress value we want to go back to our GitLab.com tab for the last time and navigate to the groups top level. We will be adding another variable so to make sure your application deployments work ENSURE you are at the group level.
1. Use the left hand navigation menu on the group level, to click through Settings > CI/CD and locate the variables section.
1. Expand the section and click Add variable.
1. Then add the variable below in addition to the existing ones making sure that protected and mask are not checked and the rest of the default settings are the same. The {your-external-ip} value is going to be the ingress ip we just copied from the gcloud console:

```
text
KUBEINGRESSBASEDOMAIN:{your-external-ip}.nip.io
```

Task 4. Deploy Ruby on Rails Application

1. Now that all of the variables are set up we should be good to go.
1. Go back to your project overview page of your group.
1. Click New project, Create from template.
1. Select and import the Ruby on Rails project with a name of your choice and public.
1. Once the project is imported use the left hand navigation menu to click through Settings > CI/CD and
1. Expand the Auto DevOps section.
1. Click Default to Auto DevOps pipeline, leave the Deployment strategy as the first option, & Save changes
1. We then want to click our project name in the top right and click Web IDE on the project homepage.
1. Within the IDE we will want to locate the .gitlab-ci.yml file and delete it.
1. Once deleted commit the changes to master and watch as the Auto DevOps pipeline uses our new agent & infra to push out an application.

> This step also makes sure that your agent was set up correctly. If your Auto DevOps pipeline does not look like the one below make sure your agent config is correct. It may be a single indent is missing that invailidated the whole yaml.

1. It is out of scope for onboarding but you could continue to modify the agent as you add more and more projects to this group. The below example is what your manifest definition might look like if you started adding multiple projects:

```
yml
gitops:
  manifestprojects:

  - id: gitlab-learn-labs/gitops/2022-10-11-ultimate-gitops/jenlung/world-greetings-env-1
    defaultnamespace: default
    paths:
      - glob: '/manifests//.yaml'
    reconciletimeout: 3600s  1 hour by default
    dryrunstrategy: none  'none' by default
    prune: true  enabled by default
    prunetimeout: 360s  1 hour by default
    prunepropagationpolicy: foreground  'foreground' by default
    inventorypolicy: mustmatch  'mustmatch' by default

  - id: gitlab-learn-labs/gitops/2022-10-11-ultimate-gitops/laidai/world-greetings-env-1
    defaultnamespace: default
    paths:
      - glob: '/manifests//.yaml'
    reconciletimeout: 3600s  1 hour by default
    dryrunstrategy: none  'none' by default
    prune: true  enabled by default
    prunetimeout: 360s  1 hour by default
    prunepropagationpolicy: foreground  'foreground' by default
    inventorypolicy: mustmatch  'mustmatch' by default
```

Sample Demo Projects

Adding Ready-To-Go Demos

- [ ] Fork one demo into your personal ultimate group

There is a shared demo group for webinars at https://gitlab.com/gitlab-learn-labs/webinars that you can fork projects out of and into your new ulitmate group & have running in under 5 minutes.

We suggest reading the documentation about how these shared projects works. These demos are great because they all come with suggested talk tracks as well for how you might present the various topics.

By forking the project, you can use them how you want and then submit MRs to the original project to collaborate with others on fixing any bugs or add any new features.

Learn more about setting up these demos at gitlab.com/gitlab-learn-labs/webinars/how-to-use-these-projects

Installing Personal GitLab Runners

- [ ] Install GitLab Runners

You only need to do this if you find yourself running out of shared runner minutes. It is typical that SAs + CSM/Es exhaust their quota of compute minutes. While installing and running an agent is an advanced topic, it is something that our customers ask a lot of questions about so it's best to be well versed on the topic. If you are brave you can follow our docs to set up the runner in a different way.

1. First we need to set up an Ubuntu VM before moving forward. Navigate to your GCP project, and click through Compute Engine > VM instances. Next click CREATE INSTANCE.

1. On the creation page add a name (something runner related), then leave the settings as is until the Boot disk section. Click Change and change the Operating system to Ubuntu & the Version 20.04 LTS x86/64, amd64. Then click SELECT.

   > If you hit "no space left on device", you can always increase the volume size at a later time

1. From there scroll down to the Firewall section and make sure both Allow HTTP traffic and Allow HTTPS traffic are selected.

1. Click CREATE and wait for the pod to spin up. Once its spun up click SHH > View gcloud command. If you have gcloud set up already on your machine copy the command and run it in your terminal of choice. If you don't, you can click RUN IN CLOUD SHELL which should auto paste in the command. Just keep in mind that the cloud shell is not reliable and you should get your local connection set up.

1. Within your terminal shell first run the below command to enter privileged mode:

```
console
sudo -i
```

1. Install Docker.

```
console
sudo apt-get update
sudo apt install docker.io
```

> If that doesn't succeed, please reference docker's documentation.

1. Install GitLab Runner.

```
console
 Add the GitLab repository
curl -L "https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh" | sudo bash
```

```
console
 Install the GitLab Runner package.
sudo apt-get install gitlab-runner
```

1. You can then check the status of the runner with this command:

```
 console
 sudo gitlab-runner status
```

1. Now that the runner is up and running we need to connect it.

- If you are configuring this at the group level, navigate to the group and using the left hand navigation menu click CI/CD > Runners. In the upper right corner, select Register a group runner and copy the registration token locally.
- If you are configuring this at the project level, navigate to the project and using the left hand navigation menu click through Settings > CICD and scroll down and expand the Runner section and copy the registration token locally.
- For more information, see the documentation for registering runners.

1. Copy this command into a text editor and replace the REGISTRATIONTOKEN placeholder below with the token that you copied locally. Copy the command into your Terminal and run it.

```
console
sudo gitlab-runner register -n \
  --url https://gitlab.com/ \
```

```
    --registration-token REGISTRATIONTOKEN \
    --executor docker \
    --description "My Docker Runner" \
    --docker-image "docker:20.10.16" \
    --docker-privileged \
    --docker-volumes "/certs/client"
```

- If you configured a group runner, navigate back to the CI/CD > Runners page in your group. You should now see your runner listed with an Online status.

> Consider modifying the Runner's configurations located at /etc/gitlab-runner/config.toml (e.g. concurrent, see here for additional details)

- If you configured a project runner, navigate back to the Settings > CICD page and expand the Runners section. You should now see your runner listed with an Online status.

Troubleshooting The Agent

- Check that your KUBECONTEXT & KUBEINGRESSBASEDOMAIN variables are configured at the group level and not the project level.

- Check that your KUBECONTEXT & KUBEINGRESSBASEDOMAIN variables do not have trailing spaces o indents

- Ensure that your agents config.yamlpoints at the right group or project

- In your gcloud shell run kubectl logs -f -l=app=gitlab-agent -n gitlab-agent to see if any errors were logged

- Check out the fagentforkubernetes Slack channel to see if the team has mentioned any ongoing issues

- Try out any further troubleshooting on this handbook page: <https://docs.gitlab.com/ee/user/clusters/agent/troubleshooting.html>

- Attend weekly office hours linked here: Demo Systems Onboarding Links

Next steps

Now that you have your own instance and projects spun up, go ahead and try out some of our customer facing workshops we put on by clicking the link below.

Please ensure that you sign up with your GitLab provided email for access. These workshop will cover the basics of Advanced CI/CD & DevSecOps & can be a good reference point for later.

Additional Notes

- If a Learn Labs Group or a group that was set up for you is mentioned, this is your group that you created above.

- Both labs have a Kubernetes Agent section that you can skip over because we have already done that.

- Recordings have been provided for you to follow along with an Instructor if you would like, but keep in mind to not try and redeem a group at the start of the recording.

See the Demo Systems Onboarding Links Google Doc for additional resources.

SECTION: customer-success/demo-systems/environments/_index.md

---
SECTION: customer-success/demo-systems/environments/training-cloud.md

Overview

The demo systems that we call the GitLab Training Cloud provides a perpetual shared GitLab instance that is used for Professional Services training classes and field marketing enablement workshops with hands-on lab scenarios. The GitLab Training Cloud is comparable to our hosted gitlab.com SaaS service, however it allows greater flexibility for demonstration and sandbox purposes without affecting our production environment.

The GitLab Training Cloud provides you access to Ultimate license features with your own user account and an organizational group that you can use for creating projects and child groups. We also handle all of the GitLab Runner autoscaling and Kubernetes configuration behind the scenes.

Workshop existing lab guide catalog usage process

This process has moved to [/handbook/customer-success/demo-systems/invitation-code-creation] and [/handbook/customer-success/demo-systems/workshop-preparation].

Workshop hands-on lab creation process

This has moved to [/handbook/customer-success/demo-systems/invitation-code-creation] and [/handbook/customer-success/demo-systems/workshop-content-creation-best-practices].

Self-service workflow for creating an invitation code

This process has moved to [/handbook/customer-success/demo-systems/invitation-code-creation] and [/handbook/customer-success/demo-systems/workshop-preparation].

Test your invitation code and sample projects

This process has moved to [/handbook/customer-success/demo-systems/invitation-code-redemption] and [/handbook/customer-success/demo-systems/workshop-preparation].

Demo Systems team scalability review

This process has been deprecated. You do not need to contact the Demo Systems team for workshop scalability reviews anymore as long as your workshop attendees are under 200 users and you have reduced your CI pipeline jobs as best you can.

Prepare for your class or workshop

This process has moved to [/handbook/customer-success/demo-systems/workshop-preparation].

During the class or workshop

This process has moved to [/handbook/customer-success/demo-systems/workshop-preparation].

After the class or workshop

This process has moved to [/handbook/customer-success/demo-systems/workshop-preparation].

Extension Policy

This process has moved to [/handbook/customer-success/demo-systems/workshop-preparation].

SECTION: customer-success/demo-systems/infrastructure/_index.md

---
SECTION: customer-success/demo-systems/infrastructure/kubernetes.md

Kubernetes Cluster Design

We have reduced the size of GKE's default cluster configuration to be right-sized for various use cases.

Small Cluster (Reservation/User Sandbox)

We're going to make an assumption that we want a small cluster to fit within a /23 CIDR range. Based on lessons learned, you quickly run out of pod IPs if you use a /24 or smaller range.

We are going to reduce the number of pods per node down to 16. This uses a /27 CIDR range for pods per node. Based on lessons learned, a fresh Kubernetes cluster uses approximately 15 pods for Kubernetes management-related containers so this ensures that a new cluster uses only 1 node.

For documentation purposes, we're going to replace the second octet of the IP addresses and CIDR ranges with x to represent an unknown variable and r to represent the /23 range allocated to a cluster. The r value will always be an even number (0, 2, 4, 6, up to 252) since a /23

consumes two /24 ranges (one even and one odd range).

CIDR Range	IP Range	Subnet Name	Capacity